Video Transcript

Video 9: Web Sockets Streaming Exercise (15:52)

So, we've been looking at web sockets and at how we can stream data from IoT and other devices. And the problem that we came up against was that the web sockets has no concept of a message. So, here we've got a server that is sending messages. Let's assume for now that the connection, the web socket connection has been set up so the server and the client are both willing to receive and send messages. The server is sending messages, and suppose it sends two 10 byte messages, so there are two sends. Now what actually goes out over the network is 20 bytes and that's going to be buffered and the client will make a read, a receive, and will get 20 bytes.

So, there is no break in the TCP packets that tells us that we're at the end of a message. We're going to have to put that in and we've got a couple of choices. One way is to have fixed length messages that we say every message is going to be 50 bytes, but that really won't work too well. So, one of the best solutions is that we're going to have a fixed length header at the beginning of a message. So, what we mean by that is we're going to choose a certain number of bytes and we can make it as long as we like, but that will be the number of characters in the message. So, we could set this to a billion and we could have a billion characters in our message.

Now we're not going to go that big, but that's the idea. So, we're going to send the message and we'll start reading through that message. And once we've read those 11 characters, we'll know we're at the end of the message. So, this is our strategy, fixed header size, and then the remainder of that will be 11 characters. So, we know we've got a certain length here, we're going to choose 11 for this length, and then the rest will be the 11 characters. Now when we get the message, it may not come to us as a single chunk, if you like.

We can decide how many bytes we want to read in our chunks. And so, we may end up with something like this that we have 31, then the Hello World, It's a good day. But then we get the beginning of the next message in that chunk. And so this is going to be an issue for us. Let's go and take a look at some code and then come back in a little more detail to this. So, here we are,

© MIT xPRO® 2024. All rights reserved.

and we've got some clients and we're going to set up first, the server that's going to send these messages.

So, we're going to choose the header for how long a message is. This is setting up the web socket, so we need to import socket. I'm importing time as well because we may want to put some sleep spaces, if you like, between these messages. And then, we're going to keep this web socket alive. So, we'll have while True and that's going to stay active for potentially forever. And we're saying we can accept connections. So, we're the server, we're going to sit there and if somebody contacts us, we'll accept that connection. We're going to send them the first message is going to be, "Hello from the Server- You are Connected".

And then we're going to have a loop where we're going to send them the time is and we're going to push the time into that message. So, what we're doing here is we need to calculate the length of the message. Now that length could be a lot of digits and we've got to make sure it's less than 10 digits because we can only go up to 10 because we chose header to be 10. So, what this is doing is putting that into a string and then padding it out to the length of the header with nothings.

So, this is going to be for example, this length of the message might be 31, so that's three and a one. So, now we've got another eight blank characters up until the end of the header, which is 10 long, and then we add the message on as a string as well. Now, we send that in bytes and using 'utf- 8' format to the client, and here we're just printing out the message so we can see it. So, here on this side, we're just zipping out a stream of messages. So, we might want to put asleep between these to slow them down. But essentially, that's what's happening. Now, let's take a look at the client side.

We need to deal with messages coming in, and we're going to decide to receive them in chunks, so we can specify how long this chunk is, how many bytes. Now the problem is that these chunks will not be related to the message lengths. So, we'll have part of a message here in the first chunk. Then we may have another part and another part, and eventually, we'll have the end of the message. So, this vertical line here, but we might have the beginning of the next message. So, here I'm showing 29, and maybe, there's a blank here because we know the header is 10 characters long.

So, this is what we've got to deal with. We're reading chunks, but we need to understand when

© MIT xPRO® 2024. All rights reserved.

the message is complete. And we need to deal with this end part and put it into the beginning of the next message. Let's take a look at our code and we start importing socket and set the header. Now we'll connect to the port on our local machine and it's 9092 on mine. You can choose, as long as you're the same on the server and the client, you'll be fine. Now, here we're reading a chunk, so we're setting up full message as blank at the moment. We're going to fill that up until we've got at least the full first message and maybe more of the next message. Let's suppose we're dealing with the first message.

So, here we read the first chunk. Now we've got to get the length from the header, the length of the message. So, I might have something already in first message, but if this is the very first message, this is going to be a blank, and we'll just have what we've read in here, that chunk. And so now, we can get the length of the message by reading from zero to the header and converting that to an integer and that'll give us the message length. So, if it's a new message, then we're going to re-get the message length.

Now, we're going to add the chunk to the full message. We're going to check if we've got the whole of that message. So, if the length of full message is greater than the message length plus header. because the header isn't in this message length, we know it's 10. But if the full message is less than that, we're going to read another chunk. But once it's greater than that so now, we're going to print out from zero up to the message length plus header, up to there. So, if the message length is 29, we're going to read up from zero to 29 plus 10, 39. Now, the start of the next message may be in that last chunk, and I'm not dealing with it here.

So, that's your challenge. You've got to deal with that next bit. So, let me show you where you we are that we are at this stage where we've got the complete message, but we may have this part. So, what we need to do is to run along in the full message up to this point and then save this. Now what do we do with that? Well, what I suggest is that we now print out this full message. So, from here to here, and now we save this next bit. Let's call it next. Whatever's left here in this chunk, we put that into the start of the next message, and we may as well use full message because we've printed out the last full message. Now,

I'm starting the next full message, so I'll put that into full message and then we'll continue reading chunks. We'll go back to the beginning again. Let me show you how the correct version runs. So, here I've got two windows. My code is in here, so I'm going to set up the server. So, we

© MIT xPRO® 2024. All rights reserved.

run stream server and that's running. Now we go and we run. You need to run your solution. I'm going to run my solution. So, stream client one solution. Let's see what happens. So, the server served up all the messages and here we are, we've caught them all. You can see that all the messages have been received by the client right up to the last digit and printed out beautifully.

So, that's what I want you to be able to do. Don't worry about these Ms. or Cs, that's me with some little bit of debugging. But what you'll need to do is to debug this code. So, this is going to throw an error. So, let me break out of this. You may have some problems with finding that when you restart that it tells you that the port is being used. But I found if you break in the client first and then in the server, do control C in the client first and then in the server, you won't have that problem.

So, I've started the server up again. Now, I'm going to run this time in debug mode here. So, I'm in your starter code and let's put a, let's put a break point in, for example, when we're we've got one there. Good. So, here we go. So, we've read in a chunk, you see it's got the 29 and it's got the rest of the header that's blank and then it's got the 'ti'. So, we've got that. Now, we need to get the length of the message. Yes, we've got 29 and that's what we've, that's fine. Now, we're going to add the incoming chunk to full message and full message is blank at the moment. But when we've added that chunk to it, now full message will be 29, the 'ti'.

Now, we're checking if the full message is longer than the message length, that's 29 plus header, that's 10, 39. Is the full message longer than that? No, it's not. So, we go back and we read another chunk. So, this is how you need to go about debugging is looking at where your code is, what it's doing, reading another chunk now. So, I've read three chunks. It's not a new message. So now, we're going to have that, this full message length. You can see it here and it looks like it might have a 30 that belongs to the next and some spaces as well. Let's take a look.

So, it certainly has come into here. We've got at least the first message printing it out there it is. Now the problem is I didn't deal with this next bit that's actually on the end of this full message. If we look at full message at the moment, notice it's got actually here that 30 and the spaces belong to the next message. I happen to know that. So now, we're going to come around, read another chunk, but so that's got the time is, but notice it doesn't have a header of any kind, it doesn't tell us the length. And that's because there's an error with my code. So, I want you to correct this. I'm giving you some hints down here about what you need to do. You don't need to

© MIT xPRO® 2024. All rights reserved.

change anything above this, but you can if you want. But make sure that you can write out the messages. But if you're in debugger, you can control the reading of the messages and step through and deal with this problem that we have the start of the next message embedded in the chunk somewhere. Okay, so good luck with this exercise. Bye for now.

© MIT xPRO® 2024. All rights reserved.